SENATI

Escuela de Tecnologías de la Información

SOLUCIONARIO

Guía 10 · Ejercicios propuestos

Aprendizaje no supervisado: Algoritmo K-Means

Material exclusivo del instructor

Carrera	Ingeniería de Software con Inteligencia Artificial
Curso	Seminario de Complementación Práctica I (PIAD-427)
Semana	10
Herramientas	Python 3 · Visual Studio Code · scikit-learn · SciPy
Instructor	Rembrandt Ubalde Enríquez

Contenido

Cómo usar este solucionario 2
Ejercicio 1. K-Means sin StandardScaler 3
Ejercicio 2. n_clusters = 3 y n_clusters = 7 5
Ejercicio 3. Práctica 03 con k = 3 7
Ejercicio 4. Dataset propio: notas de dos cursos 8
Ejercicio 5. Intervalos de confianza del 90 % y del 99 % 10
Preguntas de reflexión: respuestas modelo 12

(Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».)

Cómo usar este solucionario

Cada ejercicio trae el código completo, la salida obtenida al ejecutarlo y la explicación esperada del estudiante. Los resultados se generaron con scikit-learn 1.9 y SciPy 1.18 sobre el mismo `clientes.csv` de la Práctica 04; con otras versiones los **números de grupo** pueden cambiar, pero los perfiles y conclusiones se mantienen.

Los ejercicios 1, 2 y 5 parten de `clientes.csv`, así que el estudiante debe ejecutar antes `04_kmeans_clientes.py`.

Ejercicio 1. K-Means sin StandardScaler

Respuesta: con `clientes.csv` los centroides salen **idénticos** con y sin escalado, porque las dos columnas ya tienen escalas parecidas (desviación estándar 31.9 en ingreso y 28.8 en gasto). Si el ingreso se expresa en soles en vez de miles de soles, el resultado sin escalar se arruina: K-Means agrupa solo por ingreso y mezcla los perfiles de gasto.

Archivo: ej01_sin_escalar.py

```
# ej01_sin_escalar.py - Comparar K-Means con y sin StandardScaler
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df = pd.read_csv("clientes.csv")
print("Desviación estándar de cada columna:")
print(df.std().round(1))

# A) CON escalado (como en la Práctica 04)
esc = StandardScaler()
km_con = KMeans(n_clusters=5, n_init=10, random_state=42).fit(esc.fit_transform(df))
c_con = pd.DataFrame(esc.inverse_transform(km_con.cluster_centers_), columns=df.columns).round(1)
print("\nCentroides CON StandardScaler:")
print(c_con.sort_values("ingreso_mil").to_string(index=False))

# B) SIN escalado: K-Means trabaja directamente con los valores originales
km_sin = KMeans(n_clusters=5, n_init=10, random_state=42).fit(df)
c_sin = pd.DataFrame(km_sin.cluster_centers_, columns=df.columns).round(1)
print("\nCentroides SIN StandardScaler:")
print(c_sin.sort_values("ingreso_mil").to_string(index=False))
print(f"Inercia sin escalar: {km_sin.inertia_:.2f}")

# C) Caso extremo: ingreso en SOLES (valores miles de veces mayores que el gasto)
d = pd.DataFrame({"ingreso_soles": df["ingreso_mil"] * 1000,
                  "puntaje_gasto": df["puntaje_gasto"]})
km_s = KMeans(n_clusters=5, n_init=10, random_state=42).fit(d)
c_s = pd.DataFrame(km_s.cluster_centers_, columns=d.columns).round(1)
c_s["clientes"] = pd.Series(km_s.labels_).value_counts().sort_index().values
print("\nSIN escalar, ingreso en soles:")
print(c_s.sort_values("ingreso_soles").to_string(index=False))
```

▶ Resultado obtenido en la terminal

```
Desviación estándar: ingreso_mil 31.9 | puntaje_gasto 28.8

Centroides CON StandardScaler      Centroides SIN StandardScaler
 ingreso  gasto                     ingreso  gasto
  24.6    19.6                       24.6    19.6
  28.6    80.8                       28.6    80.8
  70.4    50.7                       70.4    50.7
  94.9    83.6                       94.9    83.6
  96.5    20.0                       96.5    20.0
Inercia sin escalar: 16257.30

SIN escalar, ingreso en soles:
 ingreso_soles  puntaje_gasto  clientes
      20776.7           33.3        30
      30030.0           60.4        50
      68937.8           51.2        37
      90386.8           49.9        38
      99775.6           52.9        45
```

Caso	Centroides	¿Grupos útiles?
Con StandardScaler	5 perfiles claros de 40 clientes cada uno	Sí

Caso	Centroides	¿Grupos útiles?
Sin escalar, ingreso en miles	Idénticos al caso anterior	Sí, por casualidad: escalas similares
Sin escalar, ingreso en soles	Gasto casi igual en todos (33 a 60); grupos de 30 a 50 clientes	No: solo separa por ingreso

Explicación esperada: K-Means usa la distancia euclidiana, así que la columna con números más grandes domina el cálculo. Con el ingreso en soles, una diferencia de 1 000 soles pesa mil veces más que una diferencia de 1 punto de gasto; por eso los clientes de «ingreso bajo, gasto alto» y «ingreso bajo, gasto bajo» terminan mezclados. La inercia sin escalar (16 257) tampoco es comparable con la escalada (18.08), porque está en otras unidades. Conclusión: escalar siempre, aunque a veces parezca no hacer falta.

> 💡 Si el estudiante solo reporta «salen iguales», la respuesta está incompleta: debe explicar **por qué** salen iguales.

Ejercicio 2. n_clusters = 3 y n_clusters = 7

Respuesta: ninguno de los dos tiene tanto sentido para el negocio como k = 5. Con k = 3 se pierden perfiles (un «súper grupo» de 120 clientes junta VIP, medios y cautelosos); con k = 7 se parten grupos reales en subgrupos que no piden estrategias distintas.

Archivo: ej02_cambiar_k.py

```
# ej02_cambiar_k.py - Comparar k = 3, 5 y 7 en la segmentación de clientes
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

df = pd.read_csv("clientes.csv")
esc = StandardScaler()
X = esc.fit_transform(df)

for k in (3, 5, 7):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    c = pd.DataFrame(esc.inverse_transform(km.cluster_centers_), columns=df.columns).round(1)
    c["clientes"] = pd.Series(km.labels_).value_counts().sort_index().values
    print(f"\n=== k = {k} | inercia {km.inertia_:.2f} | silueta {silhouette_score(X,
km.labels_):.3f}")
    print(c.sort_values(["ingreso_mil", "puntaje_gasto"]).to_string(index=False))
```

▶ Resultado obtenido en la terminal

```
=== k = 3 | inercia 133.52 | silueta 0.553
 ingreso_mil  puntaje_gasto  clientes
        24.6           19.6        40
        28.6           80.8        40
        87.3           51.4       120

=== k = 5 | inercia 18.08 | silueta 0.734
 ingreso_mil  puntaje_gasto  clientes
        24.6           19.6        40
        28.6           80.8        40
        70.4           50.7        40
        94.9           83.6        40
        96.5           20.0        40

=== k = 7 | inercia 14.26 | silueta 0.592
 ingreso_mil  puntaje_gasto  clientes
        22.6           15.2        24
        27.5           26.4        16
        28.6           80.8        40
        70.4           50.7        40
        94.9           83.6        40
        95.2           13.2        19
        97.7           26.2        21
```

k	Silueta	Qué pasa con los grupos	¿Sirve al negocio?
3	0.553	Los dos grupos de ingreso bajo se mantienen; los tres de ingreso medio-alto se funden en uno de 120 clientes con gasto promedio 51	No: el centroide (87, 51) describe a un cliente «promedio» que casi no existe; VIP y cautelosos recibirían la misma campaña
5	0.734	Cinco perfiles de 40 clientes, cada uno con estrategia propia	Sí: es la mejor opción
7	0.592	«Ingreso bajo, gasto bajo» se parte en gasto 15 y 26; «ingreso alto, gasto bajo» en gasto 13 y 26; el resto queda igual	Poco: los subgrupos difieren en unos 11 puntos de gasto y no justifican campañas distintas

Explicación esperada: al subir k la inercia siempre baja (133.52 → 18.08 → 14.26), pero eso no significa grupos mejores; la silueta sí cae fuera de k = 5. Un k pequeño **mezcla** segmentos diferentes; un k grande **fragmenta** segmentos reales. El k elegido debe dar grupos que el negocio pueda tratar de forma distinta.

Ejercicio 3. Práctica 03 con k = 3

Respuesta: tomando como centroides iniciales los puntos 0, 3 y 6, el algoritmo **converge en la iteración 2** (la iteración 1 ya encuentra los grupos definitivos y la 2 solo lo confirma). La inercia baja de 69.31 (k = 2) a 16.98, porque el grupo de la derecha se separa en «arriba» y «abajo».

Archivo: ej03_kmeans_k3.py

```
# ej03_kmeans_k3.py - K-Means paso a paso con k = 3
import numpy as np

puntos = np.array([[1, 2], [1.5, 1.8], [1, 0.6], [5, 8],
                   [8, 8], [9, 11], [8, 2], [9, 3]])
k = 3                                          # CAMBIO: 3 grupos
centroides = puntos[[0, 3, 6]].astype(float)     # CAMBIO: tercer centroide = punto (8, 2)
print("Centroides iniciales:\n", centroides)

for it in range(1, 11):
    distancias = np.linalg.norm(puntos[:, None] - centroides[None, :], axis=2)
    grupo = distancias.argmin(axis=1)
    nuevos = np.array([puntos[grupo == j].mean(axis=0) for j in range(k)])
    print(f"Iteración {it}: grupos = {grupo} | centroides = {nuevos.round(2).tolist()}")
    if np.allclose(nuevos, centroides):
        print(f"Los centroides no cambiaron: el algoritmo CONVERGIÓ en la iteración {it}.")
        break
    centroides = nuevos

# EXTRA: inercia = suma de distancias² de cada punto a su centroide
inercia = ((puntos - centroides[grupo]) ** 2).sum()
print(f"Inercia final: {inercia:.2f}")
```

▶ Resultado obtenido en la terminal

```
Centroides iniciales:
 [[1. 2.]
 [5. 8.]
 [8. 2.]]
Iteración 1: grupos = [0 0 0 1 1 1 2 2] | centroides = [[1.17, 1.47], [7.33, 9.0], [8.5, 2.5]]
Iteración 2: grupos = [0 0 0 1 1 1 2 2] | centroides = [[1.17, 1.47], [7.33, 9.0], [8.5, 2.5]]
Los centroides no cambiaron: el algoritmo CONVERGIÓ en la iteración 2.
Inercia final: 16.98
```

Grupo	Puntos	Centroide final
0	(1, 2), (1.5, 1.8), (1, 0.6)	(1.17, 1.47)
1	(5, 8), (8, 8), (9, 11)	(7.33, 9.0)
2	(8, 2), (9, 3)	(8.5, 2.5)

Análisis esperado: converge muy rápido porque los centroides iniciales ya caen uno en cada nube de puntos. El número de iteraciones depende de dónde arranquen los centroides.

💡 **Para discutir en clase:** si el estudiante elige como iniciales los puntos 0, 1 y 2 (los tres de la esquina inferior izquierda), el algoritmo tarda **3 iteraciones** y termina en una mala solución: parte en dos el grupo pequeño y junta los otros cinco puntos en uno solo (inercia 68.14, casi igual que con k = 2). Por eso scikit-learn usa **k-means++** y `n_init=10`: reparte bien los centroides iniciales y prueba varios arranques.

Ejercicio 4. Dataset propio: notas de dos cursos

Respuesta modelo: con 60 estudiantes ficticios y sus notas (escala vigesimal) de Python y Matemática, la silueta es máxima en **k = 4 (0.640)** y aparecen cuatro perfiles claros de rendimiento. Cada estudiante tendrá sus propios datos; lo que se evalúa es el procedimiento y la descripción de los grupos.

Archivo: ej04_notas.py

```
# ej04_notas.py - K-Means con un dataset propio: notas de Python y Matemática
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 1. CREAR DATOS FICTICIOS (60 estudiantes, notas de 0 a 20)
rng = np.random.default_rng(7)
perfiles = [(17, 16, 15), (15, 8, 15), (8, 15, 15), (9, 8, 15)]  # (media Python, media Mate, n)
filas = [np.column_stack([rng.normal(mp, 1.5, n), rng.normal(mm, 1.5, n)])
         for mp, mm, n in perfiles]
X = np.vstack(filas).clip(0, 20).round(1)
df = pd.DataFrame(X, columns=["nota_python", "nota_matematica"])
df.insert(0, "estudiante", [f"E{i:02d}" for i in range(1, len(df) + 1)])
df.to_csv("notas.csv", index=False)
print("Filas:", len(df))

# 2. ESCALAR Y ELEGIR k CON LA SILUETA
datos = df[["nota_python", "nota_matematica"]]
esc = StandardScaler()
Xe = esc.fit_transform(datos)
print(" k  silueta")
for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(Xe)
    print(f"{k:>2}  {silhouette_score(Xe, km.labels_):.3f}")

# 3. ENTRENAR CON EL MEJOR k E INTERPRETAR
km = KMeans(n_clusters=4, n_init=10, random_state=42).fit(Xe)
df["grupo"] = km.labels_
c = pd.DataFrame(esc.inverse_transform(km.cluster_centers_), columns=datos.columns).round(1)
c["estudiantes"] = df["grupo"].value_counts().sort_index().values
print(c)
```

▶ Resultado obtenido en la terminal

```
Filas: 60
 k  silueta
 2  0.446
 3  0.586
 4  0.640
 5  0.585
 6  0.535
   nota_python  nota_matematica  estudiantes
0         14.6              8.4           16
1          7.7             15.1           15
2         16.8             14.9           15
3          8.5              7.8           14
```

Grupo	Python	Matemática	Estudiantes	Perfil	Acción sugerida
2	16.8	14.9	15	Alto rendimiento en ambos cursos	Retos avanzados; monitores de sus compañeros
0	14.6	8.4	16	Bueno en programación, flojo en Matemática	Reforzamiento en Matemática con ejemplos en código

Grupo	Python	Matemática	Estudiantes	Perfil	Acción sugerida
1	7.7	15.1	15	Bueno en Matemática, flojo en programación	Más horas de práctica guiada en Python
3	8.5	7.8	14	Bajo en ambos cursos (en riesgo)	Tutoría y seguimiento cercano

Criterios para calificar: el dataset tiene al menos 50 filas; escala los datos; justifica k con la silueta o el codo; describe cada grupo por su centroide (no por su número) y propone una acción para cada uno.

> 💡 Si un estudiante genera datos totalmente aleatorios (sin perfiles), la silueta saldrá baja (alrededor de 0.4) y casi igual para todos los k; debe reconocer que no hay grupos naturales claros. Esa también es una conclusión válida.

Ejercicio 5. Intervalos de confianza del 90 % y del 99 %

Respuesta: a mayor nivel de confianza, **más ancho** el intervalo. Con n = 40, el IC del 90 % mide un 17 % menos que el del 95 %, y el del 99 % mide un 34 % más. La media no cambia; solo cambia el valor t que multiplica al error estándar.

Archivo: ej05_ic_niveles.py

```
# ej05_ic_niveles.py - IC del 90 %, 95 % y 99 % para el gasto promedio de cada grupo
import pandas as pd
from scipy import stats
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df = pd.read_csv("clientes.csv")
X = StandardScaler().fit_transform(df)
df["grupo"] = KMeans(n_clusters=5, n_init=10, random_state=42).fit_predict(X)

print("grupo nivel  media  lím_inf  lím_sup  ancho  t")
for g, datos in df.groupby("grupo")["puntaje_gasto"]:
    n = len(datos)
    media = datos.mean()
    ee = stats.sem(datos)                                    # s / raíz(n)
    for nivel in (0.90, 0.95, 0.99):
        li, ls = stats.t.interval(nivel, df=n - 1, loc=media, scale=ee)
        t = stats.t.ppf((1 + nivel) / 2, n - 1)              # valor de la tabla t
        print(f"{g:>4} {nivel:.0%} {media:>7.2f} {li:>8.2f} {ls:>8.2f} {ls - li:>6.2f} {t:.3f}")
```

▶ Resultado obtenido en la terminal

```
grupo nivel  media  lím_inf  lím_sup  ancho  t
   0 90%   20.00    17.91    22.09   4.19 1.685
   0 95%   20.00    17.49    22.51   5.03 2.023
   0 99%   20.00    16.63    23.37   6.73 2.708
   1 90%   80.78    79.38    82.17   2.79 1.685
   1 95%   80.78    79.10    82.45   3.35 2.023
   1 99%   80.78    78.54    83.01   4.48 2.708
   2 90%   50.67    48.99    52.36   3.37 1.685
   2 95%   50.67    48.65    52.70   4.04 2.023
   2 99%   50.67    47.97    53.38   5.41 2.708
   3 90%   19.65    17.77    21.53   3.77 1.685
   3 95%   19.65    17.39    21.91   4.52 2.023
   3 99%   19.65    16.62    22.68   6.05 2.708
   4 90%   83.65    81.82    85.48   3.67 1.685
   4 95%   83.65    81.45    85.85   4.40 2.023
   4 99%   83.65    80.70    86.60   5.89 2.708
```

Grupo	Ancho IC 90 %	Ancho IC 95 %	Ancho IC 99 %
0	4.19	5.03	6.73
1	2.79	3.35	4.48
2	3.37	4.04	5.41
3	3.77	4.52	6.05
4	3.67	4.40	5.89

Explicación esperada: el ancho es 2 · t · (s / √n). Como n y s son los mismos, solo cambia t: 1.685 (90 %), 2.023 (95 %) y 2.708 (99 %). Para estar más seguros de atrapar la media real hay que abrir más la «red»: se gana confianza y se pierde precisión. Para tener a la vez mucha confianza y un intervalo angosto, la única salida es aumentar n.

💡 **Detalle para estudiantes avanzados:** los intervalos de los grupos 1 y 4 (gasto alto) se superponen en los tres niveles (por ejemplo, al 90 %: 79.38–82.17 frente a 81.82–85.48). El grupo 2 no toca a ningún otro grupo en ningún nivel.

Preguntas de reflexión: respuestas modelo

1. **¿Por qué K-Means es no supervisado?** Porque entrena solo con las variables de entrada (X), sin etiquetas (y). Nadie le dice a qué grupo pertenece cada dato: descubre los grupos midiendo distancias, y luego nosotros interpretamos qué significa cada uno.
2. **¿Qué ventaja tiene la clasificación y cuándo solo se puede agrupar?** La clasificación puede medir su exactitud y predice clases con nombre conocido (spam, aprobado, fraude). El agrupamiento es la única opción cuando no hay etiquetas o etiquetar sería muy caro: segmentar clientes nuevos, explorar datos que nadie ha estudiado o encontrar patrones que aún no tienen nombre.
3. **¿Por qué hay que escalar antes de K-Means?** Porque K-Means usa la distancia euclidiana y la columna con valores más grandes domina el cálculo. Sin escalar, ingresos en miles de soles aplastan a una edad o un puntaje de 1 a 100 (ver Ejercicio 1). StandardScaler deja todas las columnas con media 0 y desviación 1 para que pesen igual.
4. **¿Qué significa un IC del 95 %?** Que el método usado acierta el 95 % de las veces: si repitiéramos el muestreo muchas veces, 95 de cada 100 intervalos contendrían la media real de la población. No significa que la media tenga 95 % de probabilidad de estar en un intervalo ya calculado.

> 💡 Respuesta incorrecta frecuente en la pregunta 4: «el 95 % de los datos está dentro del intervalo». El IC habla de la **media**, no de los datos individuales.